

# PROGRAMACIÓN AVANZADA

Interfaz de Usuario  
(GUI) Swing



Ing. Matias Macrino

# APLICACIONES BASADAS EN ENTORNO GRAFICO (GUI)

- ▶ GUI, su significado es, Interfaz Grafica de Usuario, proviene de su nombre en ingles Graphical User Interface.  
Es preferida por el usuario, ya que es mas intuitiva.
- ▶ Conlleva una programación mas compleja, por su manejo de los distintos componentes GUI.  
El desarrollo de dichas aplicaciones se encuentra orienta a **eventos**.  
Como funcionan:
  - El usuario clickea sobre un componente.
  - Este genere un señal.
  - El programa lo captura por el escuchador (listener).
  - Ejecuta los procedimientos y algoritmos correspondientes.

# APLICACIONES BASADAS EN ENTORNO GRAFICO (GUI)

- ▶ Para aplicaciones de escritorio Java incluye como parte de sus propias librerías dos conjuntos de ellas, que proporcionan una amplia variedad de componentes gráficos. Estas son:
  - ▶ AWT (Abstract Windows Toolkit)
  - ▶ Swing (incorporado en su versión 1.2)
- ▶ Los nombres de los componentes de Swing comienzan con J  
Ejemplo Button en AWT y JButton en Swing
- ▶ Los componentes de AWT se encuentran dentro de “java.awt”
- ▶ Los componentes de Swing se encuentran dentro de “javax.swing”

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- ▶ Se presenta como una mejora de AWT, entre sus mejoras se destaca por:
  - ▶ Apariencia independiente de la plataforma (S.O)
  - ▶ Mayor numero de componentes gráficos.
  - ▶ Mejora de los componentes clásicos.



# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- ▶ Para la creación mas rápida y sencilla de una interfaz grafica, existe diversos plugins o aplicaciones que generan dicha interfaz y exportan el código Java para su utilización.
- ▶ Un plugin conocido para el IDE eclipse, es “**Window Builder**”.

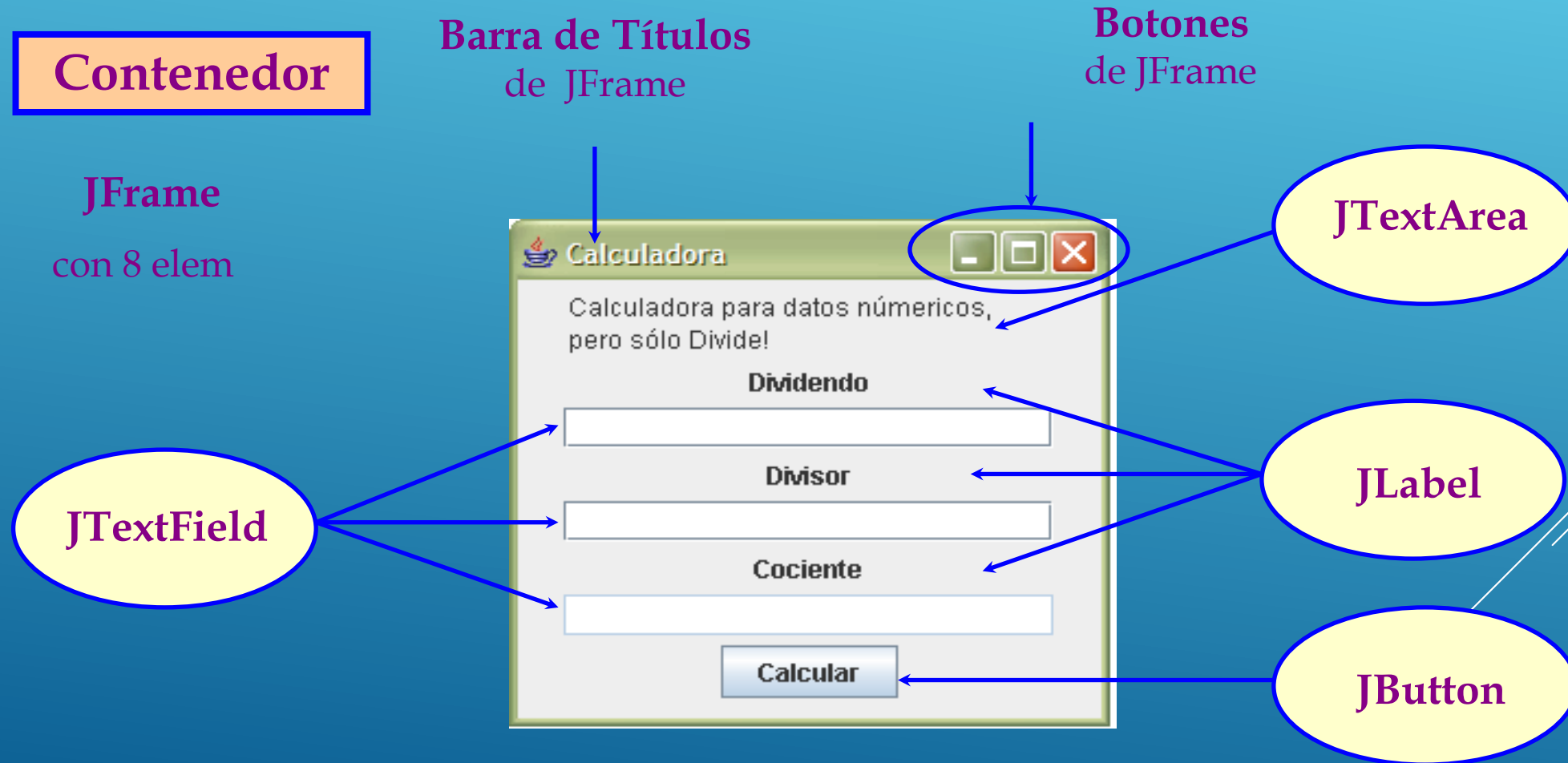
# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- Componentes: elementos que componen una interfaz grafica.

|            |              |
|------------|--------------|
| JFrame     | JCheckBox    |
| JTextArea  | JRadioButton |
| JLabel     | JComboBox    |
| JTextField | JList        |
| JButton    | JPanel       |
| JMenuBar   | JPopupMenu   |

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## ► Componentes



# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- ▶ Programación dirigida por “Eventos”:

- ▶ La programación dirigida por eventos: es el paradigma de ejecución de la programación gráfica.
- ▶ Manipulación de eventos: mecanismo para procesar cualquier evento.
- ▶ Modelo de eventos basado en delegación:
  - ▶ Objetos de fuentes de eventos
  - ▶ Objetos de recepción de eventos
- ▶ Los receptores de eventos deben ser asociados o registrados a los generadores.

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- ▶ Programación dirigida por “Eventos”:

- ▶ Un evento generado sin registro de recepción alguno, es ignorado.
- ▶ Cada tipo de evento, tiene un receptor correspondiente.
  - ▶ Receptores de acción para eventos de acción
  - ▶ Receptores de ventana para eventos de ventana
  - ▶ Receptores de mouse para eventos de mouse
- ▶ Un solo receptor se puede asociar a múltiples fuentes de eventos y viceversa, aunque no suele ser tan común.

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- ▶ Programación dirigida por “Eventos”:

- ▶ El control de flujo de un programa para consola y el de programa con interfaz gráfica muestra similitudes y diferencias.
- ▶ Programa para consola:
  - ▶ Empieza y termina en su método main()
  - ▶ Las sentencias se ejecutan en orden

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

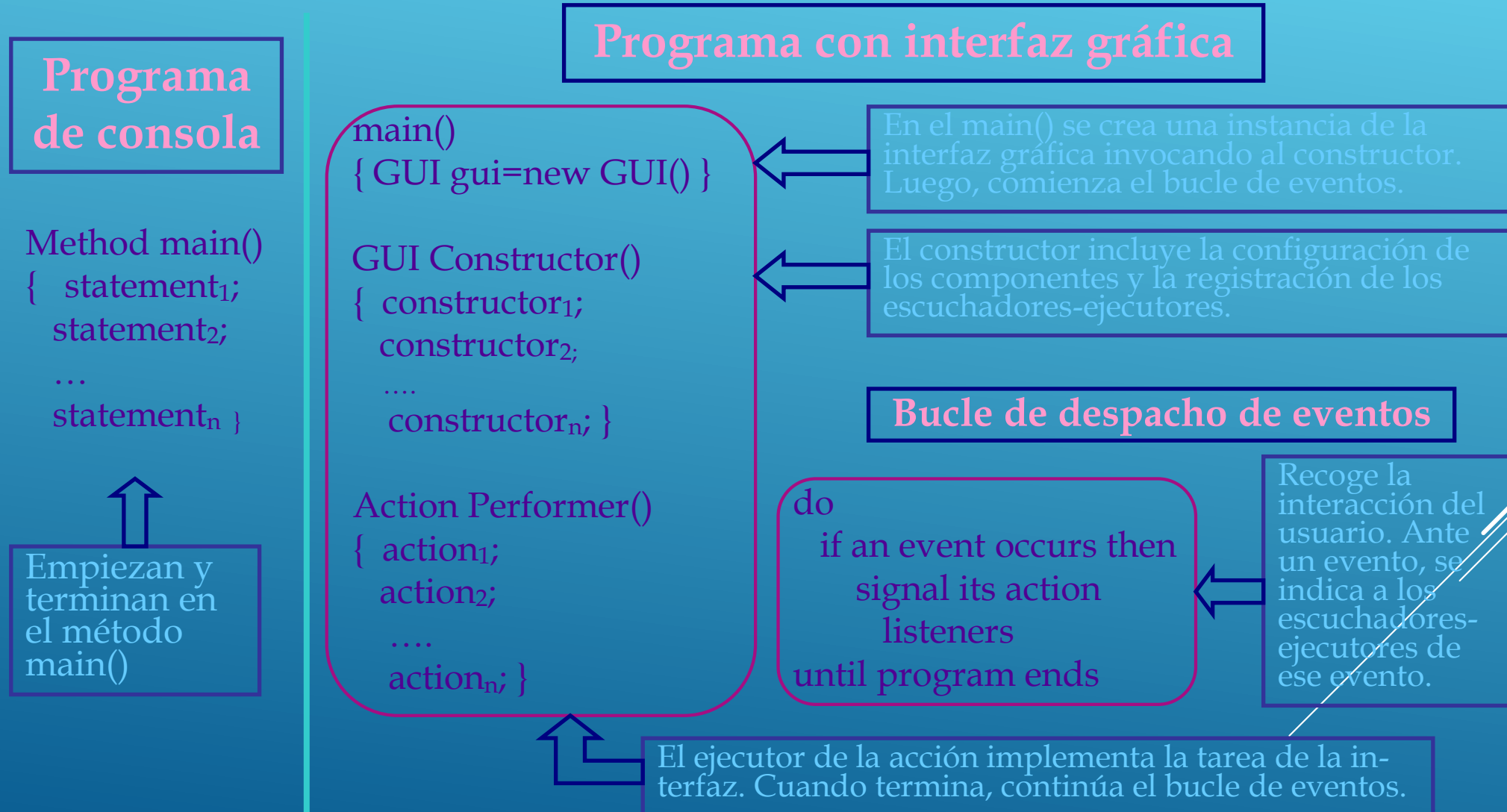
- ▶ Programación dirigida por “Eventos”:

- ▶ Programa con interfaz gráfica:

- ▶ Comienza en el método `main()`, para crear una instancia de un bucle de despacho de evento, comprueba si se producen iteraciones de usuarios (eventos de acción).
  - ▶ Evento: se llama a un método ejecutor de acción para cada escuchador de dicho evento.
  - ▶ El método procesa la interacción.
  - ▶ Después de despachar el evento, el flujo de control vuelve al bucle original, en espera de nuevos eventos.
- 
- Several white lines of varying lengths and orientations are drawn in the bottom right corner of the slide, creating a modern, abstract graphic element.

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## ► Programación dirigida por "Eventos":



# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## CONTENEDORES - CONTAINER

Estos son componentes que permiten almacenar, alojar o contener otros componentes. Swing provee algunos contenedores útiles para diferentes casos. Los mas comunes de ellos son:

**JFrame** (Es el contenedor principal y mas común utilizado)

**JDialog** (Similar a JFrame, solo que representa una ventan de dialogo)

**JPanel** (Es el contenedor en donde se alojan todos los componentes)

**JScrollPane** (Permite vincular barras de desplazamiento para los componentes)

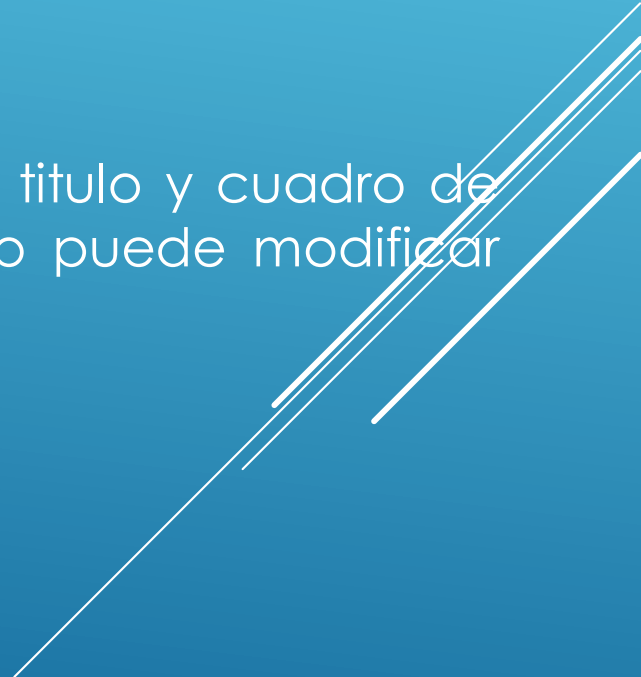
**JDesktopPane** (Este contenedor solo aloja componentes del tipo JInternalFrame)

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## CONTENEDORES - JFRAME

Este contenedor, es una subclase de Window, por lo tanto, agregan funcionalidad al comportamiento de su clase base.

Su aspecto es el de la clásica ventana de Windows, con barra de título y cuadro de control para maximizar, minimizar y cerrar. Normalmente, el usuario puede modificar manualmente el tamaño y la posición de las ventanas.

Several thin, white, parallel diagonal lines are drawn across the bottom right corner of the slide, extending from the right edge towards the bottom left.

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- ▶ Los controles swing no se añaden directamente al objeto “JFrame”. Dicho objeto incluye un contenedor interno, el cual es conocido como “panel de contenido”, donde se deben situar los componentes visuales de la interfaz.
  - ▶ Container: `this.getContentPane();`

```
public class Ventana extends JFrame {  
  
    public Ventana() {  
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        this.getContentPane().add(new JLabel("Hola Mundo"));  
        this.setBounds(100, 100, 450, 300);  
    }  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

- ▶ `DO_NOTHING_ON_CLOSE`: No realiza acción.
- ▶ `HIDE_ON_CLOSE` (default) oculta el frame cuando el usuario lo cierra pero no se deshace de los recursos del sistema asociados (puedo volver a mostrar).
- ▶ `DISPOSE_ON_CLOSE` oculta el frame y llama al método `dispose()`, para liberar recursos.
- ▶ `EXIT_ON_CLOSE`, cierra la aplicación equivale a **`System.exit(0)`**

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## CONTENEDORES - JPANEL

Estos contenedores no disponen de barra de titulo ni cuadro de control, además, su tamaño y posición son fijos.

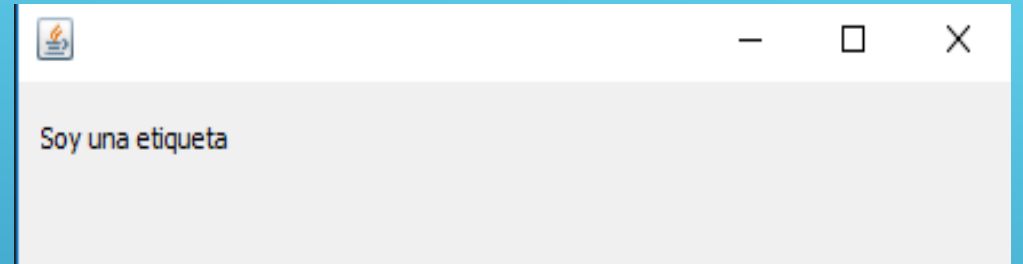
Por lo general, un panel esta a su vez contenido en otro tipo de contenedor, que puede ser una marco (JFrame).

Permiten una mejor distribución y agrupación de los componentes, además de permitir una configuración específica para cada una de esas agrupaciones.

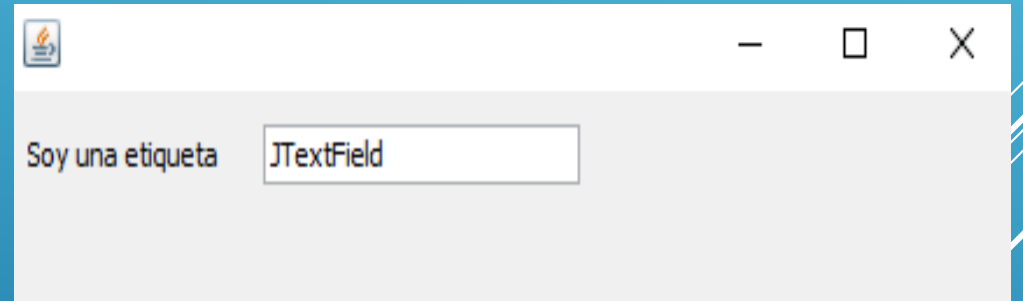
# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## COMPONENTES

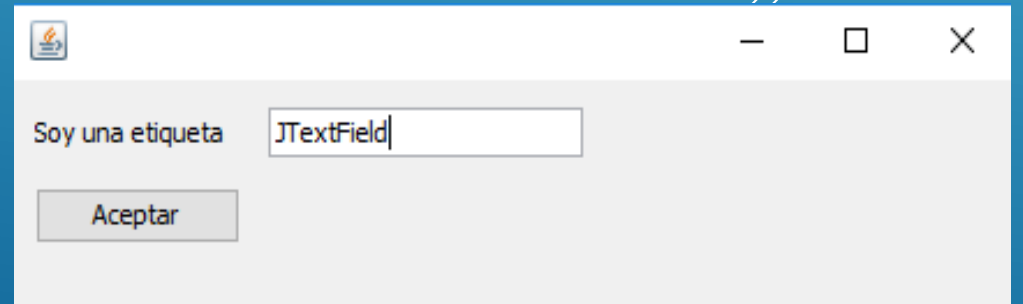
**JLabel:** Este componente sirve para insertar etiquetas, las cuales el usuario no podrá modificar. (JLabel etiqueta = new JLabel("Soy una etiqueta");)



**TextField:** Este componente es una caja de texto, en donde se le podrá pedir información a ingresar al usuario. Seria el equivalente a la clase Scanner para aplicaciones en consola (JTextField cajaTexto = new JTextField();)



**Button:** Este componente es el clásico boton de toda aplicación basada en entorno grafico. Este mismo siempre al ser pulsado dispara un evento. (JButton boton = new JButton("Aceptar");)

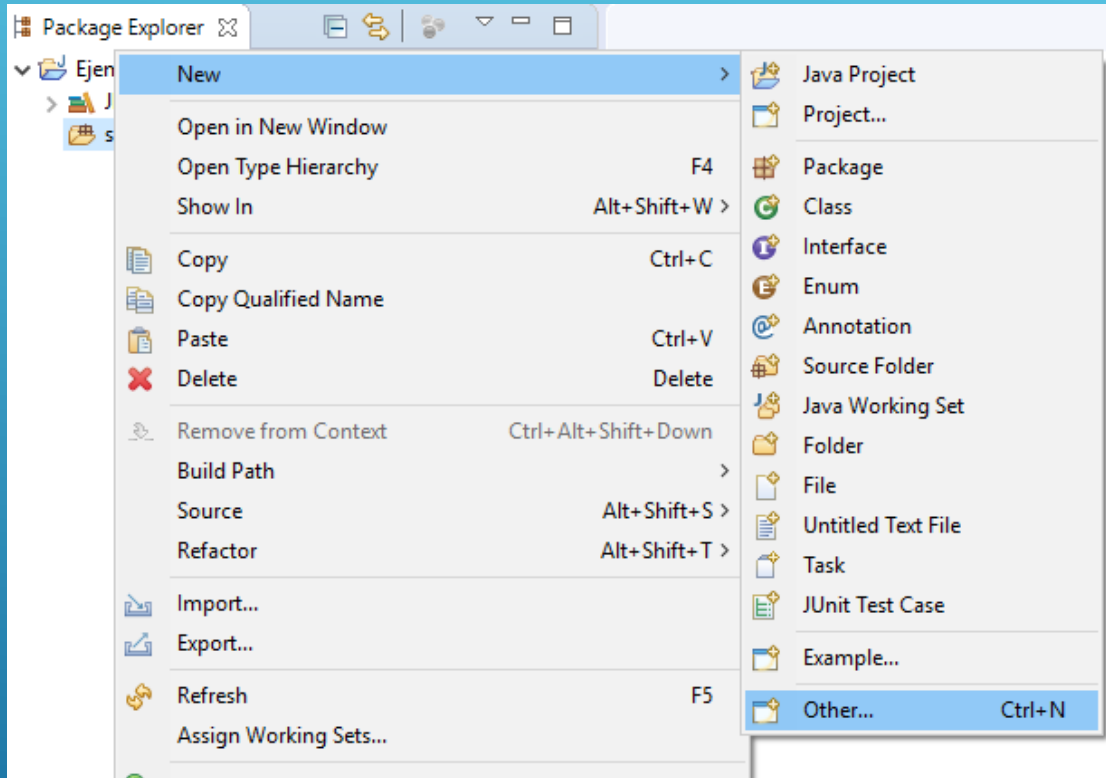


APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

***PRIMER VENTA EN SWING  
CON WINDOW BUILDER***

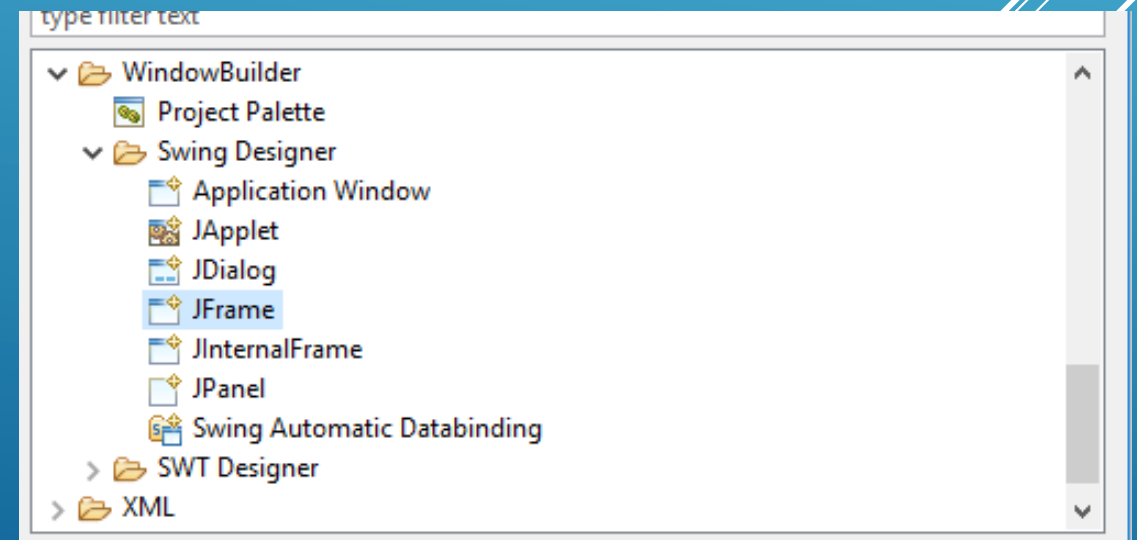
Several thin, parallel white lines are drawn diagonally across the bottom right corner of the slide, adding a modern, geometric touch to the design.

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)



1 - Hacer clic derecho sobre **src** -> **New** -> **Other**

2 - **WindowBuilder** -> **Swing Designer** -> **JFrame**



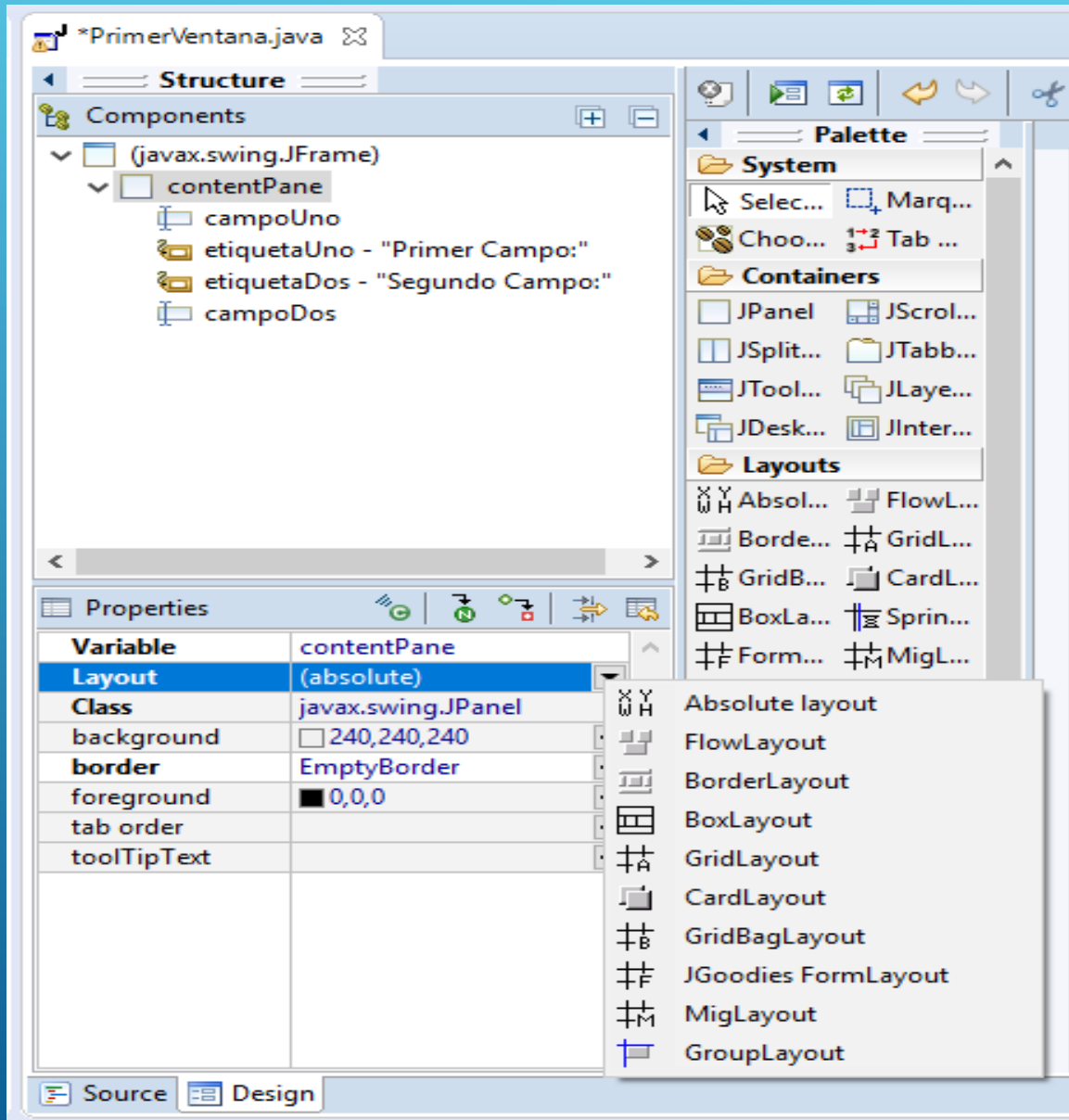
# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

3 – Completar campo **Name:**,  
equivale al nombre de la clase.

|                                                                               |                    |           |
|-------------------------------------------------------------------------------|--------------------|-----------|
| Source folder:                                                                | EjemploSwing/src   | Browse... |
| Package:                                                                      | edu.usal.ejemplo   | Browse... |
| Name:                                                                         | PrimerVentana      |           |
| Superclass:                                                                   | javax.swing.JFrame | Browse... |
| <input checked="" type="checkbox"/> Use advanced template for generate JFrame |                    |           |

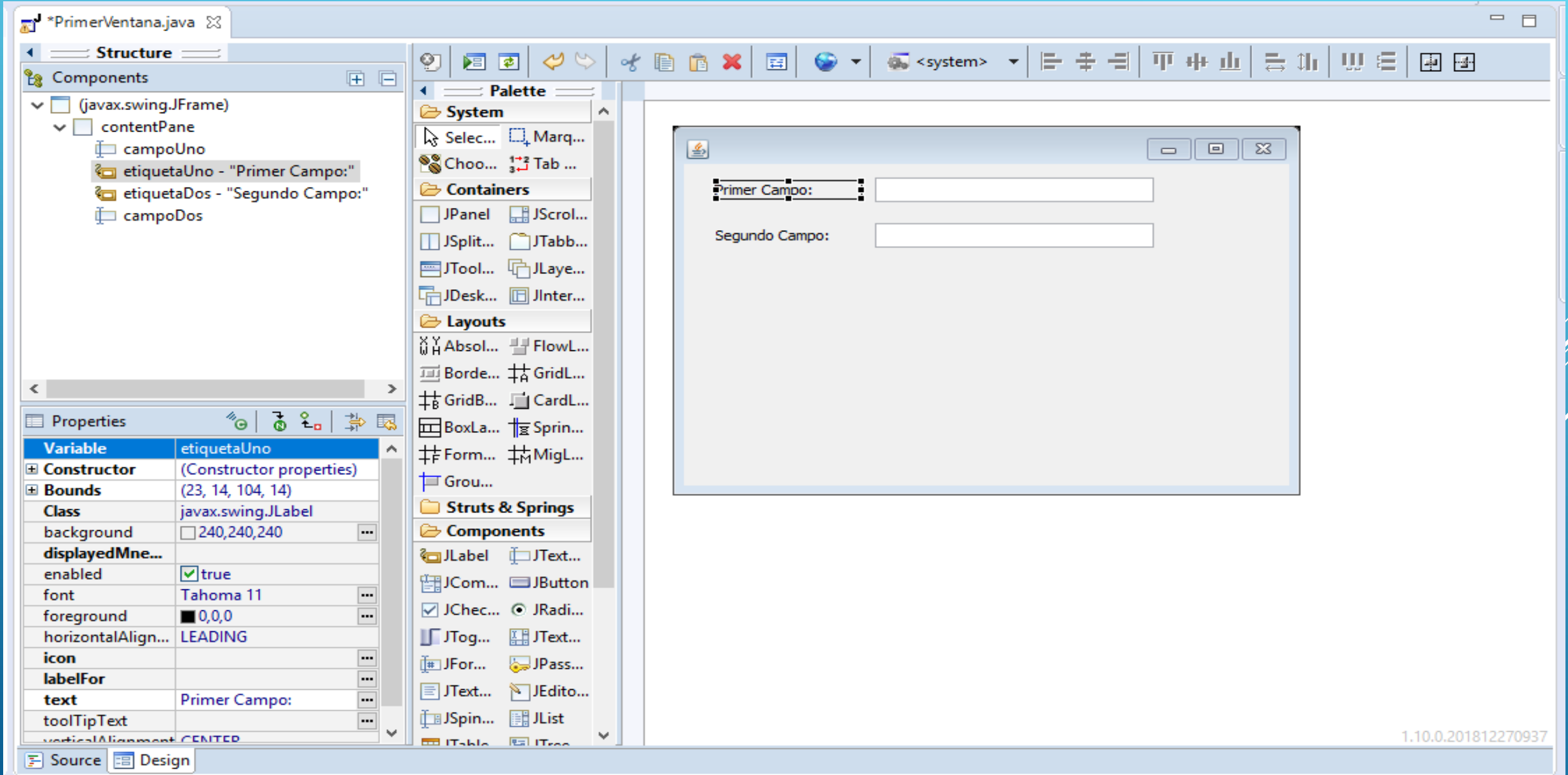
```
PrimerVentana.java
1 package edu.usal.ejemplo;
2
3 import java.awt.BorderLayout;
4
5
6
7
8
9
10 public class PrimerVentana extends JFrame {
11
12     private JPanel contentPane;
13
14     /**
15      * Launch the application.
16      */
17     public static void main(String[] args) {
18         EventQueue.invokeLater(new Runnable() {
19             public void run() {
20                 try {
21                     PrimerVentana frame = new PrimerVentana();
22                     frame.setVisible(true);
23                 } catch (Exception e) {
24                     e.printStackTrace();
25                 }
26             }
27         });
28     }
29
30     /**
31      * Create the frame.
32      */
33     public PrimerVentana() {
34         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
35     }
36 }
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)



Ante de comenzar a arrojar componentes sobre el panel (***contentPane***), tendremos que setear la propiedad ***Layout*** en ***absolute***

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)



# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Al ir a la pestaña **Source** estaremos viendo el código autogenerated por el plugin

```
*/  
public PrimerVentana() {  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setBounds(100, 100, 450, 300);  
    contentPane = new JPanel();  
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));  
    setContentPane(contentPane);  
    contentPane.setLayout(null);  
  
    campoUno = new JTextField();  
    campoUno.setBounds(137, 11, 200, 20);  
    contentPane.add(campoUno);  
    campoUno.setColumns(10);  
  
    JLabel etiquetaUno = new JLabel("Primer Campo:");  
    etiquetaUno.setBounds(23, 14, 104, 14);  
    contentPane.add(etiquetaUno);  
  
    JLabel etiquetaDos = new JLabel("Segundo Campo:");  
    etiquetaDos.setBounds(23, 51, 104, 14);  
    contentPane.add(etiquetaDos);  
  
    campoDos = new JTextField();  
    campoDos.setColumns(10);  
    campoDos.setBounds(137, 48, 200, 20);  
    contentPane.add(campoDos);  
}  
}
```

# PROGRAMACIÓN AVANZADA

Interfaz de Usuario  
(GUI) Swing – Eventos



Ing. Matias Macrino

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

**Programación dirigida por eventos:** este es el paradigma de ejecución de la programación grafica.

Cada tipo de evento, tiene un receptor correspondiente como “**receptores de acción para eventos de acción**”, “**receptores de mouse para eventos de mouse**”, etc.

**Interfaz ActionListener:** los eventos son manipulados por los componentes de dicha interfaz.

Dicha interfaz nos obliga a implementar el método **actionPerformed()**.

El mismo recibe un parámetro que es un objeto de la clase **ActionEvent**.

Con el método **getSource()** de la clase **ActionEvent** podemos saber que componente produjo el evento, ya que es lo que retorna.

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

**Método *ActionPerformed*:** dicho método se puede implementar de varias formas.

Caso 1: La clase principal implementa ActionListener.

Ejemplo: registro de botón con ActionListener

```
boton.addActionListener(this);  
public void actionPerformed(ActionEvent e) {  
    [CODIGO]  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 1: La clase principal implementa ActionListener.*

-> Declarar la clase principal del programa (esta clase si implementa la interfaz **ActionListener**):

Ejemplo: **class Action\_Listener1 implements ActionListener { .... }**

-> Registrar el componente con la propia clase a que pertenece (ya que implementa ActionListener):

Ejemplo: **componente.addActionListener(this);**

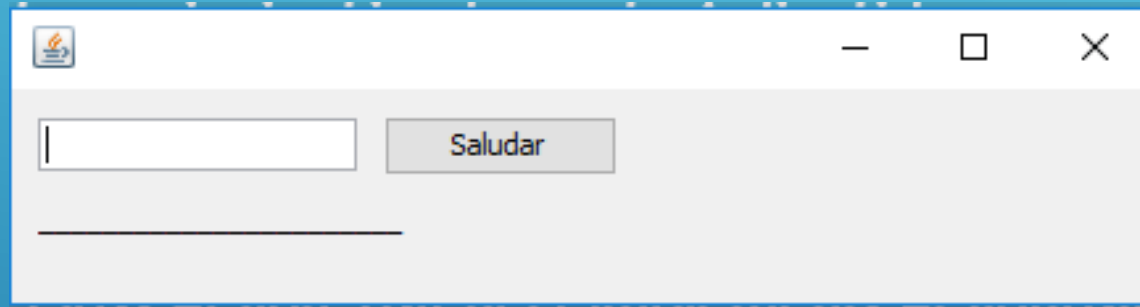
-> El método **actionPerformed** contiene las instrucciones que ejecutan la acción deseada.

Ejemplo: **public void actionPerformed(ActionEvent e) { ... }**

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 1: La clase principal implementa ActionListener.*



### 1er Paso:

```
public class CasoUno extends JFrame implements ActionListener{  
  
    private JPanel contentPane;  
    private JTextField txtSaludo;  
    private JButton btnSaludar;  
    private JLabel lblSaludo;
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 1: La clase principal implementa ActionListener.*

*2do Paso:*

```
btnSaludar = new JButton("Saludar");  
btnSaludar.setBounds(141, 10, 89, 23);  
btnSaludar.addActionListener(this);  
contentPane.add(btnSaludar);
```

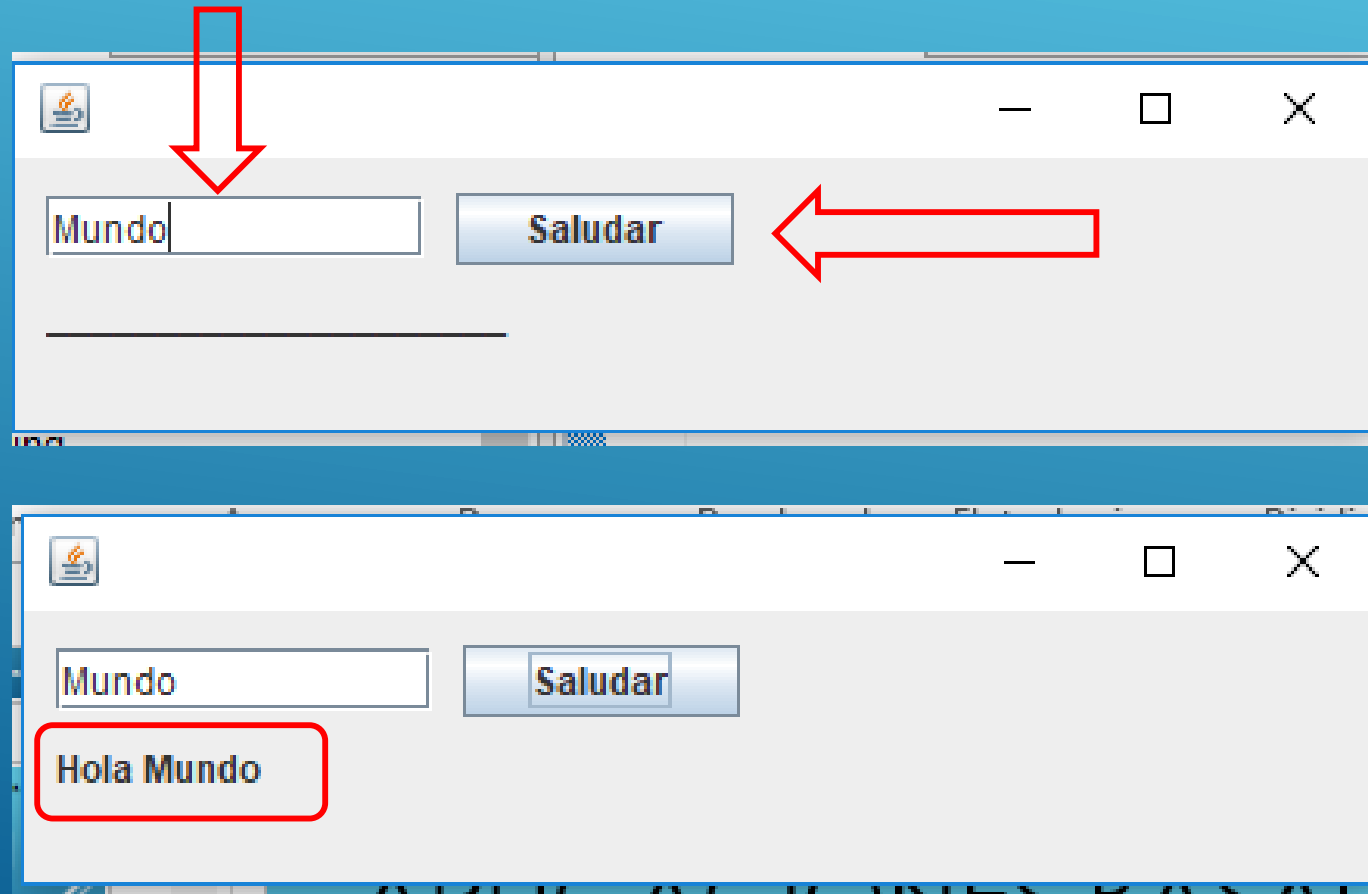
*3er Paso:*

```
@Override  
public void actionPerformed(ActionEvent arg0) {  
    if(arg0.getSource() == btnSaludar) {  
        lblSaludo.setText("Hola " + txtSaludo.getText());  
    }  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 1: La clase principal implementa ActionListener.*



# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

Caso 2: La clase principal no implementa ActionListener pero si lo hace otra clase.  
Ejemplo: registro de botón con ActionListener asociado a una nueva instancia de esta clase.

```
boton.addActionListener(new ClaseClicEnBoton());  
class ClaseClicEnBoton implements ActionListener {  
    public void actionPerformed(ActionEvent e) {  
        }  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 2: La clase principal no implementa ActionListener pero si lo hace otra clase.*

-> Declarar la clase principal del programa (esta clase **no implementa** la interfaz ActionListener).

Ejemplo: **class Action\_Listener1 { ... }**

-> Crear una nueva clase implementando ActionListener.

Ejemplo: **class ClaseClicEnBoton implements ActionListener {**  
                  **public void actionPerformed (ActionEvent e) { ... }**  
                  **}**

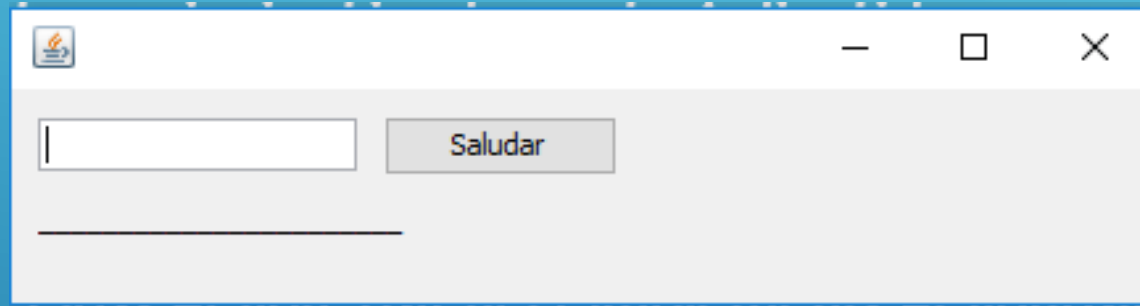
-> Registrar el componente con una instancia de la clase ClaseClicEnBoton:

Ejemplo: **componente.addActionListener( new ClaseClicEnBoton);**

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 2: La clase principal no implementa ActionListener pero si lo hace otra clase.*



*1er Paso:*

```
public class CasoDos extends JFrame {  
  
    public JPanel contentPane;  
    public JTextField textField;  
    public JButton button;  
    public JLabel label;  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 2: La clase principal no implementa ActionListener pero si lo hace otra clase.*

2do Paso:

```
public class ClicEnBoton implements ActionListener{

    private CasoDos casoDos;

    public ClicEnBoton(CasoDos casoDos) {
        this.casoDos = casoDos;
    }

    @Override
    public void actionPerformed(ActionEvent arg0) {
        if(arg0.getSource() == this.casoDos.button){
            this.casoDos.label.setText("Hola " + this.casoDos.textField.getText());
        }
    }

}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

*Caso 2: La clase principal no implementa ActionListener pero si lo hace otra clase.*

*3er Paso:*

```
button = new JButton("Saludar");  
button.setBounds(141, 11, 89, 23);  
button.addActionListener(new ClicEnBoton(this));  
contentPane.add(button);
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

Caso 3: Otra instancia implementa ActionListener

Ejemplo: registro de botón con ActionListener asociado a una nueva instancia de la clase ActionListener

```
boton.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        clicEnBotn(e)  
    }  
}  
  
private void clicEnBoton(ActionEvent e) { ... }
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

### Caso 3: Otra instancia implementa ActionListener

-> Declarar la clase principal del programa (esta clase no implementa la interfaz ActionListener):

Ejemplo: **`class action_Listener1 { ... }`**

-> Registrar el botón con una instancia de la Interface ActionListener:

Ejemplo: **`boton.addActionListener(new ActionListener()`**

**`{ public void actionPerformed(ActionEvent e)`**

**`{ clicEnBoton(e); } });`**

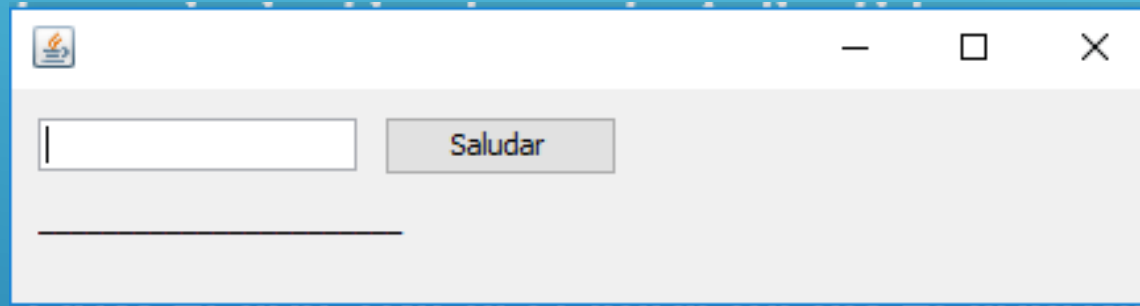
-> El método clicEnBoton contiene las instrucciones que ejecutan la acción deseada:

Ejemplo: **`private void clicEnBoton(ActionEvent evt) { ... código ... ; }`**

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

### Caso 3: Otra instancia implementa ActionListener



#### 1er Paso:

```
public class CasoTres extends JFrame {  
  
    private JPanel contentPane;  
    private JTextField textField;  
    private JButton button;  
    private JLabel label;  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

### Caso 3: Otra instancia implementa ActionListener

#### 2do Paso:

```
button.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        clicEnBoton(e);  
    }  
});  
contentPane.add(button);
```

#### 3er Paso:

```
private void clicEnBoton(ActionEvent e) {  
    label.setText("Hola " + textField.getText());  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

Caso 4: Múltiples eventos.

En caso de tener que detectar el botón que origina un evento entre varios botones, recurrir al método **setActionCommand**.

```
boton1.setActionCommand("nombre_ActionCommand1");  
boton2.setActionCommand("nombre_ActionCommand2");
```

El método **getActionCommand**, devuelve el nombre asignado:

```
public void actionPerformed(ActionEvent e) {  
    if ("nombre_ActionCommand1".equals(e.getActionCommand()))  
        { ... código ... ; }  
    else if ("nombre_ActionCommand2".equals(e.getActionCommand()))  
        { ... código ... ; } } }
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

Caso 4: Múltiples eventos.

-> Declarar la clase principal del programa (esta clase sí implementa la interfaz ActionListener):

Ejemplo: **`class action_Listener1 implements ActionListener { ... }`**

-> Configurar la propiedad **ActionCommand** de los botones:

Ejemplo: **`boton1.setActionCommand ("nombre_ActionCommand1");`**  
**`boton2.setActionCommand ("nombre_ActionCommand2");`**

-> Registrar los botones con la propia clase a que pertenecen (implementa **ActionListener**):

Ejemplo: **`boton1.addActionListener(this);`**  
**`boton2.addActionListener(this);`**

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

Caso 4: Múltiples eventos.

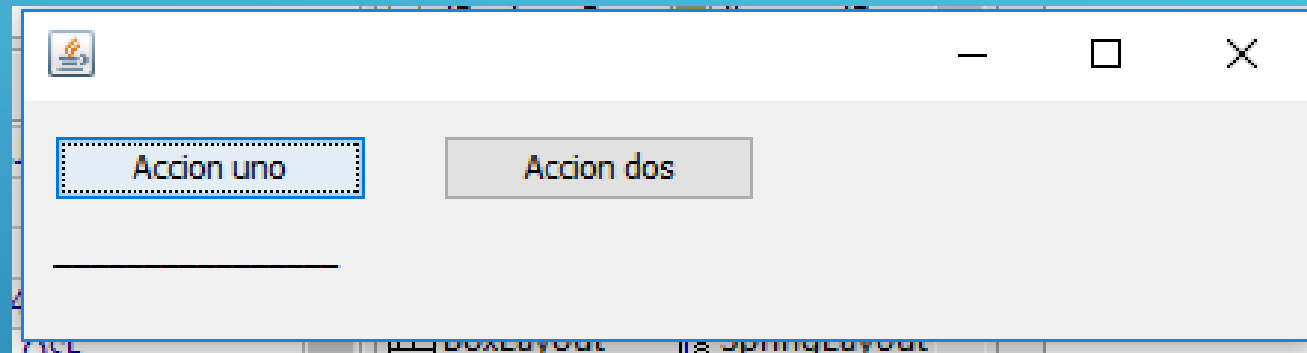
-> Recuperar el valor de la propiedad **ActionCommand** para saber cuál es el botón que ha generado el evento:

```
Ejemplo: public void actionPerformed(ActionEvent e) {  
    if ("nombre_ActionCommand1".equals(e.getActionCommand()))  
        {...código...;}  
    else if ("nombre_ActionCommand2".equals(e.getActionCommand()))  
        {...código...; }  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

### Caso 4: Múltiples eventos.



#### 1er Paso:

```
public class CasoCuatro extends JFrame implements ActionListener{  
  
    private JPanel contentPane;  
    private JButton btnAccionDos;  
    private JButton btnAccionUno;  
    private JLabel label;  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

### Caso 4: Múltiples eventos.

#### 2do Paso:

```
btnAccionDos = new JButton("Accion dos");  
btnAccionDos.setBounds(141, 11, 106, 23);  
btnAccionDos.setActionCommand("A2");  
contentPane.add(btnAccionDos);  
  
btnAccionUno = new JButton("Accion uno");  
btnAccionUno.setBounds(10, 11, 106, 23);  
btnAccionUno.setActionCommand("A1");  
contentPane.add(btnAccionUno);
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

### Caso 4: Múltiples eventos.

#### 3er Paso:

```
btnAccionDos.addActionListener(this);  
btnAccionUno.addActionListener(this);
```

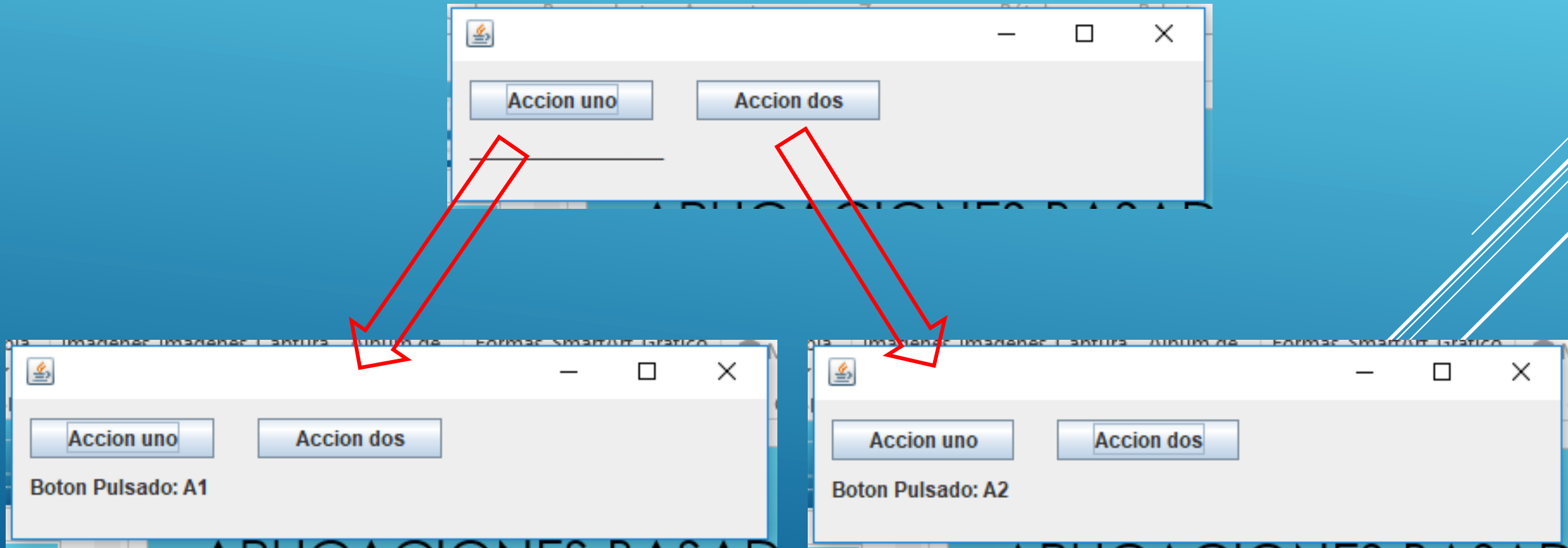
#### 4to Paso:

```
@Override  
public void actionPerformed(ActionEvent e) {  
    if("A1".equals(e.getActionCommand())) {  
        label.setText("Boton Pulsado: " + e.getActionCommand());  
    } else if("A2".equals(e.getActionCommand())) {  
        label.setText("Boton Pulsado: " + e.getActionCommand());  
    }  
}
```

# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## EVENTOS

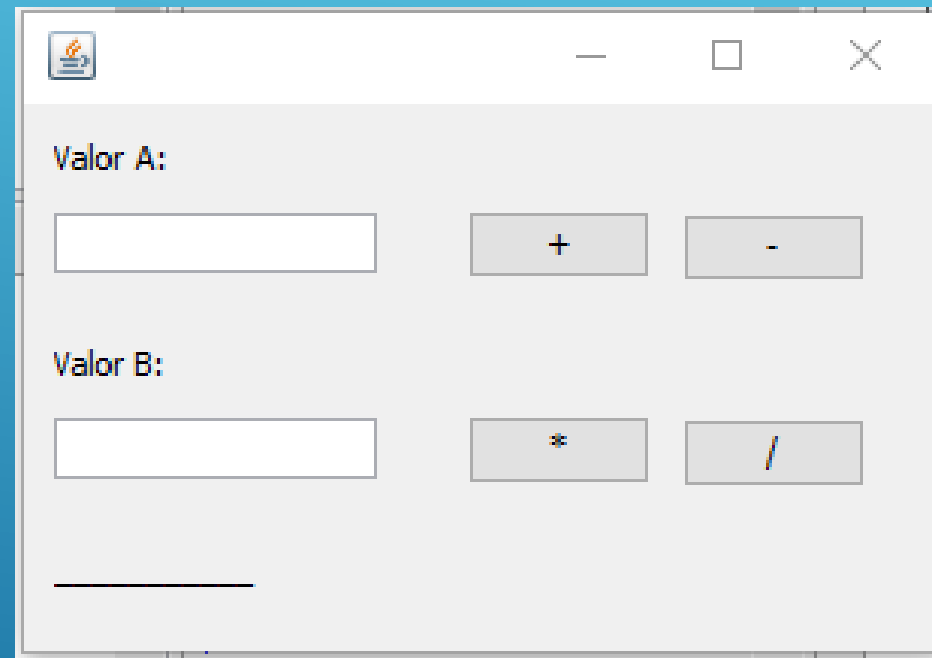
### Caso 4: Múltiples eventos.



# APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

## *EVENTOS - Ejercicio*

### Calculadora básica:



- *Usar caso dos para manejo de eventos, los componentes no deben de ser de acceso publico.*
- *Uso de objetos para realizar los cálculos.*